Number of Ways to Reorder Array to Get Same BST

Problem

You are given an array nums that represents a permutation of integers from 1 to n. We are going to construct a binary search tree (BST) by inserting the elements of nums in order into an initially empty BST. Find the number of different ways to reorder nums so that the constructed BST is identical to that formed from the original array nums.

Return the number of ways to reorder nums such that the BST formed is identical to the original BST formed from nums. Since the answer may be very large, return it modulo 10^9 + 7.

Example 1:
Input: nums = [2,1,3]
Output: 1
Explanation: We can reorder nums to be [2,3,1] which will yield the same BST. There are no other ways.

Example 2:
Input: nums = [3,4,5,1,2]
Output: 5

Example 3:
Input: nums = [1,2,3]
Output: 0

Constraints:
1 <= nums.length <= 1000
1 <= nums[i] <= nums.length
All integers in nums are distinct.

Approach

Build the BST conceptually from nums, then count valid interleavings recursively.

For a subtree with L values in its left subtree and R values in its right subtree, the number of ways to interleave them while preserving each subtree's internal order is:

C(L + R, L)

The recurrence is:

ways(node) = C(L + R, L) * ways(left) * ways(right)

The root must always appear first, so the final answer is:

ways(nums) - 1

Time complexity: O(n^2)
Space complexity: O(n^2) for the binomial coefficient table.

Java Solution

```java
import java.util.ArrayList;
import java.util.List;

class Solution {
    private static final int MOD = 1_000_000_007;
    private long[][] combinations;

    public int numOfWays(int[] nums) {
        int n = nums.length;
        combinations = new long[n + 1][n + 1];

        for (int i = 0; i <= n; i++) {
            combinations[i][0] = 1;

            for (int j = 1; j <= i; j++) {
                combinations[i][j] =
                    (combinations[i - 1][j - 1] + combinations[i - 1][j]) % MOD;
            }
        }

        List<Integer> values = new ArrayList<>();
        for (int num : nums) {
            values.add(num);
        }

        long total = countWays(values);
        return (int) ((total - 1 + MOD) % MOD);
    }

    private long countWays(List<Integer> values) {
        if (values.size() <= 2) {
            return 1;
        }

        int root = values.get(0);
        List<Integer> left = new ArrayList<>();
        List<Integer> right = new ArrayList<>();

        for (int i = 1; i < values.size(); i++) {
            int value = values.get(i);

            if (value < root) {
                left.add(value);
            } else {
                right.add(value);
            }
        }

        long leftWays = countWays(left);
        long rightWays = countWays(right);

        long interleavings =
            combinations[left.size() + right.size()][left.size()];

        return leftWays * rightWays % MOD * interleavings % MOD;
    }
}
```